Esse ai resolve com CCW

Thomaz Miranda, Miguel Batista, Joao Arthur Marques

February 27, 2026

Contents

1 Data Structures 1
- 1.1 Fenwick Tree (Binary Indexed Tree) 1
- 1.2 Segment Tree (Range Query + Point Update) 1
- 1.3 Segment Tree (Range Query + Range Update) 1
- 1.4 Sparse Table (Idempotent Range Query) 2
- 1.5 Disjoint Set Union (Union-Find) 2
- 1.6 Implicit Treap (Sequence Treap) 2
- 1.7 Order-Statistic Tree (PBDS) 2
- 1.8 Convex Hull Trick Dinamico 2

2 Graphs 3
- 2.1 Dijkstra's Shortest Paths 3
- 2.2 Dinic's Maximum Flow (with Scaling) 3
- 2.3 Min Cost Max Flow (MCMF) 3
- 2.4 Bridge Detection (Tarjan) 4
- 2.5 Strongly Connected Components (Kosaraju) 4
- 2.6 2-SAT 4
- 2.7 Topological Sort (Kahn's Algorithm) 5
- 2.8 LCA com binary lifting 5
- 2.9 Heavy Light Decomposition (Path query + Path update) 5
- 2.10 Centroid Decomposition - pares a distancia k 5

3 Math 6
- 3.1 Modular Integer 6
- 3.2 Modular Arithmetic Helpers 6
- 3.3 Linear Sieve (prime precomputation) 6
- 3.4 FFT/NTT Convolution 6
- 3.5 Matrix template 7
- 3.6 Gauss 7

4 Strings 7
- 4.1 Knuth-Morris-Pratt (KMP) 7
- 4.2 Aho-Corasick Automaton 8
- 4.3 Suffix Array - O(n log n) 8

5 Dynamic Programming 8
- 5.1 Divide and Conquer DP Optimization 8
- 5.2 Sum over Subsets DP (SOS DP) 8

6 Geometry 8
- 6.1 Integer Geometry Primitives 8
- 6.2 Float Geometry Primitives 9
- 6.3 Convex Hull (Monotone Chain) 10
- 6.4 Halfplane Intersection 10

7 Extra 10
- 7.1 template 10
- 7.2 Pragmas 10

1 Data Structures

1.1 Fenwick Tree (Binary Indexed Tree) 5cd

Supports point updates and prefix/range sum queries in logarithmic time using a 1-indexed BIT.

Complexity: O(log N) per op, O(N)

```
e7f struct Bit {
5c1     ll n;
fa4     v64 bit;
1cd     Bit(ll _n = 0) : n(_n), bit(n + 1) {}
bd3     Bit(v64& v) : n(sz(v)), bit(n + 1) {
3a1         for (ll i = 1; i <= n; i++) {
289             bit[i] += v[i - 1];
481             ll j = i + (i & -i);
b3c             if (j <= n) bit[j] += bit[i];
631         }
e3d     }
adf     void update(ll i, ll x) { // soma x na posicao i
7fa         for (i++; i <= n; i += i & -i) bit[i] += x;
53a     }
8e6     ll pref(ll i) { // soma [0, i]
125         ll ret = 0;
8fb         for (i++; i; i -= i & -i) ret += bit[i];
3cb         return ret;
4bb     }
a76     ll query(ll l, ll r) {  // soma [l, r]
745         return pref(r) - pref(l - 1);
9a9     }
a32     ll upper_bound(ll x) {
fc8         ll p = 0;
3b6         for (ll i = __lg(n); i+1; i--)
8f5             if (p + (1<<i) <= n and bit[p + (1<<i)] <= x)
218                 x -= bit[p += (1 << i)];
4e5         return p;
f6d     }
5cd };
```

1.2 Segment Tree (Range Query + Point Update) 04c

supports point updates and range queries.

Complexity: O(log N) per op, O(N)

```
5a4 struct node {
42e     ll val = 0;

c31     static node comb(const node& a, const node& b) {
5b6         return {a.val + b.val};
e5f     }
b99 };

52f template<typename T> struct segtree {
5c1   ll n;
141   T neutral;
8fd   vector<T> tree;

a96   segtree<T>(ll _n, T _neutral = T()) {
384     n = _n, neutral = _neutral;
19d     tree.resize(2*n+1, neutral);
d85   }

ab6   void set_leaves(vector<T> &leaves) {
ba7     copy(leaves.begin(), leaves.end(), tree.begin() + n);

da6     for (ll i = n - 1; i > 0; i--) tree[i] = T::comb(tree[2 * i],
 tree[2 * i + 1]);
1c0   }

a99   void update(ll i, T v) {
2c1     i += n;
440     tree[i] = v;
9fc     while (i > 1) {
b73       i /= 2;
a5b       tree[i] = T::comb(tree[2 * i], tree[2 * i + 1]);
cbd     }
9be   }

894   T query(ll i, ll j) {
bf2     T rl = neutral, rr = neutral;
2b8     for(i += n, j += n; i <= j; i /= 2, j /= 2){
40f       if((i&1) == 1) rl = T::comb(rl, tree[i++]);
0e9       if((j&1) == 0) rr = T::comb(tree[j--], rr);
933     }
5e7     return T::comb(rl, rr);
d69   }
32e };
```

1.3 Segment Tree (Range Query + Range Update) bb4

Tree for range queries with a customizable combine; supports range updates and range queries.

Complexity: O(log N) per op, O(N) to build

```
cd2 struct Node {
cbd     ll sum, cnt;
54d     Node operator*(const Node &o) const { return {sum + o.sum,
cnt + o.cnt}; }
135 };

9f2 struct Update {
23e     ll add = 0;
997     optional<ll> set;

02c     Node operator()(const Node &n) const {
13d         ll res = set.has_value() ? *set * n.cnt : n.sum;
4c9         return {res + add * n.cnt, n.cnt};
9d0     }

919     Update operator+(const Update &o) const {
5f4         Update res = *this;
a36         if (o.set.has_value()) {
f1e             res.set = o.set;
9f5             res.add = 0;
18c         }
7fe         if (o.add != 0) {
fa9             if (res.set.has_value()) *res.set += o.add;
63b             else res.add += o.add;
e1c         }
8df         return res;
407     }
dad };

58d template<typename T, typename U> struct segtree {
471   ll s, h;

808   T id;
377   vector<T> val;

a3e   U noop;
d47   vector<bool> dirty;
65b   vector<U> prop;

f5d   explicit segtree(ll ts, T tid = T(), U tnoop = U()) {
11b     id = tid, noop = tnoop;
836     for (s = 1, h = 1; s < ts; ) s *= 2, h++;

8e7     val.assign(2*s, id);
4a5     dirty.assign(2*s, false);
363     prop.assign(2*s, noop);
0b3   }

6db   void set_leaves(vector<T> &lvs) {
a5c     copy(lvs.begin(), lvs.end(), val.begin() + s);
```

```
95f     for (ll i = s - 1; i > 0; i--) val[i] = val[2 * i] * val[2 *
 i + 1];
4a5     dirty.assign(2*s, false);
363     prop.assign(2*s, noop);
3a5   }

570   void apply(ll i, U &upd) {
564     val[i] = upd(val[i]);
c4c     if(i < s) {
675       prop[i] = prop[i] + upd;
2d8       dirty[i] = true;
2ca     }
703   }

30b   void pull(ll i) {
fa9     for (ll l = i/2; l; l /= 2) {
c7a       T comb = val[2*l] * val[2*l+1];
a08       val[l] = prop[l](comb);
325     }
eee   }

540   void push(ll i) {
c2d     for (ll th = h; th > 0; th--) {
dcd       ll l = i >> th;

38a       if (dirty[l]) {
4fb         apply(2*l, prop[l]);
fd5         apply(2*l+1, prop[l]);

6ad         prop[l] = noop;
7b3         dirty[l] = false;
983       }
db3     }
7b6   }

41b   void update(ll i, ll j, U upd) {
33a     i += s, j += s;
f9d     push(i), push(j);

2a7     for (ll l = i, r = j; l <= r; l /= 2, r /= 2) {
6a1       if((l&1) == 1) apply(l++, upd);
7a4       if((r&1) == 0) apply(r--, upd);
f70     }

cf2     pull(i), pull(j);
470   }

894   T query(ll i, ll j){
33a     i += s, j += s;
f9d     push(i), push(j);

487     T rl = id, rr = id;
810     for(; i <= j; i /= 2, j /= 2){
2bc       if((i&1) == 1) rl = rl * val[i++];
03c       if((j&1) == 0) rr = val[j--] * rr;
4a8     }
f0e     return rl * rr;
b69   }
cd7 };
```

1.4 Sparse Table (Idempotent Range Query) 191

Preprocesses static array to answer idempotent range queries (e.g., min/max) in O(1) after O(N log N) build.

Complexity: O(N log N) build, O(1) query; O(N log N) space

```
549 struct sparse {
d87     vector<v64> m;

7bf     sparse(v64& v) {
3d3         ll n = sz(v);
571         ll logn = 64 - __builtin_clzll(n - 1);
260         m.resize(n+1, v64(logn+1));

725         forn(i, 0, n) m[i][0] = v[i];

2eb         for (ll j = 1; (1 << j) <= n; j++)
f0e         for (ll i = 0; i + (1 << j) <= n; i++)
4fe             m[i][j] = min(m[i][j-1], m[i + (1 << (j-1))][j-1]);
780     }

cb2     ll query(ll a, ll b) {
19f         ll j = __builtin_clzll(1) - __builtin_clzll(b - a + 1);
966         return min(m[a][j], m[b - (1 << j) + 1][j]);
525     }
191 };
```

1.5 Disjoint Set Union (Union-Find) 55e

Supports find with path compression and union by size to maintain dynamic connectivity of disjoint sets.

Complexity: O(alpha(N)) amortized per op, O(N)

```
da2 struct dsu {
c4c     v64 id, len;

b10
    dsu(ll n) : id(n), len(n, 1) { iota(id.begin(), id.end(), 0); }

15e
    ll find(ll a) { return a == id[a] ? a : id[a] = find(id[a]); }

847     bool uni(ll a, ll b) {
f78         a = find(a), b = find(b);
6c5         if (a == b) return false;
940         if (len[a] < len[b]) swap(a, b);
3b0         len[a] += len[b], id[b] = a;
01d         return true;
433     }
55e };
```

1.6 Implicit Treap (Sequence Treap) 0a3

Maintains a sequence with split and merge operations using randomized priorities and subtree sizes.

Complexity: O(log N) expected per op, O(N)

```
bcd struct Treap{
091     ll val;
549     ll prio, size;
b09     vector<Treap*> kids;
e02     Treap(ll c): val(c), prio(rand()), size(1),
bc6         kids({NULL,NULL}){};
2f4 };

0c2 ll size(Treap *me){return me ? me->size : 0;}

ae2 void rsz(Treap* me){me -> size =
5bf     1 + size(me->kids[0]) + size(me->kids[1]);}

5d0 vector<Treap*> split(Treap *me, ll idx){
79a     if(!me) return {NULL,NULL};
161     vector<Treap*> out;

9b0     if(size(me->kids[0]) < idx){
863         auto aux = split(me->kids[1],
4b0             idx - size(me->kids[0]) -1);
58e         me->kids[1] = aux[0];
3b0         rsz(me);
853         out = {me, aux[1]};
8cc     }else{
72d         auto aux = split(me->kids[0], idx);
f69         me->kids[0] = aux[1];
3b0         rsz(me);
d01         out = {aux[0], me};
26e     }
b44     return out;
683 }

a3f Treap* merge(Treap *left, Treap *right){
b53     if(left == NULL) return right;
03c     if(right == NULL) return left;

b29     Treap* out;

994     if(left->prio < right->prio){
1e6         left->kids[1] = merge(left->kids[1], right);
a81         rsz(left);
3a5         out = left;
b7a     }else{
5e8         right->kids[0] = merge(left, right->kids[0]);
4c9         rsz(right);
93a         out = right;
2d7     }
b44     return out;
629 }
```

1.7 Order-Statistic Tree (PBDS) 468

Wraps __gnu_pbds tree to support order_of_key and find_by_order operations on a sorted set.

Complexity: O(log N) per op, O(N)

```
663 #include <ext/pb_ds/assoc_container.hpp>
8ea #include <ext/pb_ds/tree_policy.hpp>

df9 using namespace __gnu_pbds;

99c #define ordered_set tree<p64, null_type,less<p64>, rb_tree_tag,
tree_order_statistics_node_update>

8c2 int main() {
fe6     ordered_set s;
dbe     s.find_by_order(position);
ab6     s.order_of_key(value);
731 }
```

1.8 Convex Hull Trick Dinamico 66a

para double, use INF = 1/.0, div(a, b) = a/b update(x) atualiza o ponto de intersecao da reta x overlap(x) verifica se a reta x sobrepoe a proxima add(a, b) adiciona reta

da forma ax + b query(x) computa maximo de ax + b para entre as retas O(log(n)) amortizado por insercao O(log(n)) por query

```
5c0 struct Line {
647     mutable ll a, b, p;
f15     bool operator<(const Line& o) const { return a < o.a; }
22f     bool operator<(ll x) const { return p < x; }
7f5 };

8a7 struct CHT : multiset<Line, less<>> {
9cb     ll div(ll a, ll b) {
941         return a / b - ((a ^ b) < 0 && a % b);
63f     }

956     void update(iterator x) {
c7b         if (next(x) == end()) x->p = INF;
36b         else if (x->a == next(x)->a) x->p = x->b >= next(x)->b ?
 INF : -INF;
558         else x->p = div(next(x)->b - x->b, x->a - next(x)->a);
d23     }

cf0     bool overlap(iterator x) {
9b4         update(x);
54b         if (next(x) == end()) return 0;
b2f         if (x->a == next(x)->a) return x->b >= next(x)->b;
2cd         return x->p >= next(x)->p;
136     }

1a7     void add(ll a, ll b) {
395         auto x = insert({a, b, 0});
67e         while (overlap(x)) erase(next(x)), update(x);
5f6         if (x != begin() && !overlap(prev(x))) x = prev(x),
update(x);
af3         while (x != begin() && overlap(prev(x)))
c8a             x = prev(x), erase(next(x)), update(x);
8e8     }

1f7     ll query(ll x) {
84a         assert(!empty());
1e0         auto l = *lower_bound(x);
8d4 #warning cuidado com overflow!
6a8         return l.a * x + l.b;
e51     }
287 };
```

2 Graphs

2.1 Dijkstra's Shortest Paths acf

Computes single-source shortest paths on non-negative weighted graphs using a priority queue.

Complexity: O((N + M) log N), O(N + M)

```
d2a vector<vector<p64>> g;

// d = distance | p = from/path
72f void dijkstra(ll s, v64 &d, v64& p) {
7ed     ll n = sz(g);
7c5     d.assign(n, INF);
c87     p.assign(n, -1);

480     d[s] = 0;
d8d     priority_queue<p64> pq;
33e     pq.push({0, s});
09f     while (!pq.empty()) {
27e         ll u = pq.top().second;
311         ll d_u = -pq.top().first;
6a5         pq.pop();

46d         if (d_u != d[u]) continue;

f1f         for (auto edge : g[u]) {
8a2             ll v = edge.first;
549             ll w_v = edge.second;

f4f             if (d[u] + w_v < d[v]) {
0eb                 d[v] = d[u] + w_v;
1ca                 p[v] = u;
305                 pq.push({-d[v], v});
f3f             }
841         }
e12     }
582 }
```

2.2 Dinic's Maximum Flow (with Scaling) 012

Computes max flow using Dinic's algorithm with optional capacity scaling to speed up BFS levels.

Complexity: O(E V$\hat{}${}2) worst-case, O(E)

```
7c5 struct dinitz {
85b     const bool scaling = true;
16e     ll lim;
60e     struct edge {
914         ll to, cap, rev, flow;
1c9         bool res;
6ff         edge(ll to_, ll cap_, ll rev_, bool res_)
973
            : to(to_), cap(cap_), rev(rev_), flow(0), res(res_) {}
4d0     };

b0e     vector<vector<edge>> g;
c60     v64 lev, beg;
f30     ll F;
e9f     dinitz(ll n) : g(n), F(0) {}

fd8     void add(ll a, ll b, ll c) {
1ae         g[a].emplace_back(b, c, sz(g[b]), false);
3be         g[b].emplace_back(a, 0, sz(g[a])-1, true);
b9c     }

c3f     bool bfs(ll s, ll t) {
09a         lev = v64(sz(g), -1); lev[s] = 0;
f7f         beg = v64(sz(g), 0);
71c         queue<ll> q; q.push(s);
dad         while (sz(q)) {
9c6             ll u = q.front(); q.pop();
7af             for (auto& i : g[u]) {
099
                if (lev[i.to] != -1 || (i.flow == i.cap)) continue;
c7b                 if (scaling && i.cap - i.flow < lim) continue;
d86                 lev[i.to] = lev[u] + 1;
138                 q.push(i.to);
52f             }
c49         }
498         return lev[t] != -1;
544     }
797     ll dfs(ll v, ll s, ll f = INF) {
9e3         if (!f || v == s) return f;
be3         for (ll& i = beg[v]; i < sz(g[v]); i++) {
29f             auto& e = g[v][i];
9dd             if (lev[e.to] != lev[v] + 1) continue;
985             ll foi = dfs(e.to, s, min(f, e.cap - e.flow));
789             if (!foi) continue;
f4f             e.flow += foi, g[e.to][e.rev].flow -= foi;
a79             return foi;
080         }
ca4         return 0;
207     }

f34     ll max_flow(ll s, ll t) {
9f7         for (lim = scaling ? (1<<30) : 1; lim; lim /= 2)
3e3             while (bfs(s, t)) while (ll ff = dfs(s, t)) F += ff;
1fa         return F;
bf6     }

0e5     void reset() {
19b         F = 0;
74a         for (auto& edges : g) for (auto& e : edges) e.flow = 0;
50f     }
012 };
```

2.3 Min Cost Max Flow (MCMF) 10d

min_cost_flow(s, t, f) computes the pair (flow, cost) where max(flow) <= f with min(cost). min_cost_flow(s, t) -> Maximum flow with minimum cost from s to t. For DAGs, SPFA can be replaced with DP to avoid O(nm) initial cost. If no negative cost edges exist, SPFA is not needed.

Complexity: O(nm + f * m log n), O(n + m)

```
4ad template<typename T> struct mcmf {
60e     struct edge {
500         ll to, rev, flow, cap; // destination, reverse edge id,
current flow, capacity
1c9         bool res; // is reverse edge
ccb         T cost; // cost per unit of flow
6d3         edge() : to(0), rev(0), flow(0), cap(0), cost(0), res(
false) {}
6db
        edge(ll to_, ll rev_, ll flow_, ll cap_, T cost_, bool res_)
883             : to(to_), rev(rev_), flow(flow_), cap(cap_), res(
res_), cost(cost_) {}
8e9     };

b0e     vector<vector<edge>> g;
c97     v64 par_idx, par;
6f5     T inf;
cdb     vector<T> dist;

9aa     mcmf(ll n) : g(n), par_idx(n), par(n), inf(numeric_limits<T
>::max()/3) {}

45f     void add(ll u, ll v, ll w, T cost) { // edge from u to v
with capacity w and cost
b6f         edge a = edge(v, sz(g[v]), 0, w, cost, false);
6db         edge b = edge(u, sz(g[u]), 0, 0, -cost, true);

4c5         g[u].push_back(a);
6c1         g[v].push_back(b);
72a     }
```

```
5cd
    vector<T> spfa(ll s) { // not needed if no negative cost edges
6d1         deque<ll> q;
f30         vector<bool> is_inside(sz(g), 0);
f73         dist = vector<T>(sz(g), inf);

81e         dist[s] = 0;
261         q.push_back(s);
ded         is_inside[s] = true;

b27         while (!q.empty()) {
bf4             ll v = q.front();
be6             q.pop_front();
f1e             is_inside[v] = false;

b24             forn(i,0,sz(g[v])){
7ef                 auto [to, rev, flow, cap, res, cost] = g[v][i];
446                 if (flow < cap && dist[v] + cost < dist[to]) {
60c                     dist[to] = dist[v] + cost;

166                     if (is_inside[to]) continue;
ff5                     if (!q.empty() && dist[to] > dist[q.front()])
 q.push_back(to);
7d3                     else q.push_front(to);
4ed                     is_inside[to] = true;
ceb                 }
d67             }
ef5         }
c9d         return dist;
48a     }
0e9     bool dijkstra(ll s, ll t, vector<T>& pot) {
c2f         priority_queue<pair<T, ll>, vector<pair<T, ll>>, greater
<>> q;
f73         dist = vector<T>(sz(g), inf);
81e         dist[s] = 0;
aa9         q.emplace(0, s);
dad         while (sz(q)) {
7df             auto [d, v] = q.top();
f93             q.pop();
e90             if (dist[v] < d) continue;
b24             forn(i,0,sz(g[v])) {
7ef                 auto [to, rev, flow, cap, res, cost] = g[v][i];
3c0                 cost += pot[v] - pot[to];
446                 if (flow < cap && dist[v] + cost < dist[to]) {
60c                     dist[to] = dist[v] + cost;
f90                     q.emplace(dist[to], to);
921                     par_idx[to] = i, par[to] = v;
633                 }
fb9             }
af9         }
98b         return dist[t] < inf;
b13     }

8c4     pair<ll, T> min_cost_flow(ll s, ll t, ll flow = INF) {
947         vector<T> pot(sz(g), 0);
da6         pot = spfa(s); // change shortest path algorithm here
         // no negative costs: nothing needed
         // DAG: use DP instead
d3a         ll f = 0;
8a4         T ret = 0;
8cf         while (f < flow && dijkstra(s, t, pot)) {
afe             forn(i,0,sz(g))
85f                 if (dist[i] < inf) pot[i] += dist[i];

dda             ll mn_flow = flow - f, u = t;
c1a             while (u != s){
715                 mn_flow = min(mn_flow,
582                     g[par[u]][par_idx[u]].cap - g[par[u]][
par_idx[u]].flow);
a09                 u = par[u];
27c             }

50a             ret += pot[t] * mn_flow;

060             u = t;
c1a             while (u != s) {
5b9                 g[par[u]][par_idx[u]].flow += mn_flow;
e52                 g[u][g[par[u]][par_idx[u]].rev].flow -= mn_flow;
a09                 u = par[u];
47d             }

335             f += mn_flow;
350         }

b5a         return make_pair(f, ret);
e41     }

     // Optional: returns original edges where flow == cap
5d9     vector<p64> recover() {
0b1         vector<p64> used;
7c2         forn(i, 0, sz(g)) for (edge e : g[i])
e76
            if(e.flow == e.cap && !e.res) used.push_back({i, e.to});
992         return used;
997     }
10d };
```

2.4 Bridge Detection (Tarjan) c79

Finds all bridges in an undirected graph via DFS timestamps and low-link values.
Complexity: O(N + M), O(N + M)

```
cf5 vector<v64> g;
ed6 vector<bool> visited;
09a vector<ll> tin, low;
540 ll timer = 0;

100 void dfs(ll u, ll p = -1) {
0fe     visited[u] = true;
f0e     tin[u] = low[u] = timer++;
e1b     for (ll v : g[u]) {
85a         if (v == p) continue;

4f6         if (visited[v]) {
7a0             low[u] = min(low[u], tin[v]);
254         } else {
4c8             dfs(v, u);
890             low[u] = min(low[u], low[v]);
367             if (low[v] > tin[u]) {
                 // THIS IS A BRIDGE
cd7             }
020         }
0e8     }
a89 }

ff6 void find_bridges() {
8c5     timer = 0;
e18     visited.assign(n, false);
5fa     tin.assign(n, -1);
25d     low.assign(n, -1);
bda     forn(i, 0, n) {
eed         if (!visited[i])
275             dfs(i);
341     }
706 }
```

2.5 Strongly Connected Components (Kosaraju) af8

Computes SCCs using two DFS passes and builds the condensation graph.
Complexity: O(N + M), O(N + M)

```
ed6 vector<bool> visited;

be2 void dfs(ll v, vector<v64>& g, vector<ll> &out) {
9dc     visited[v] = true;
619     for(auto u : g[v]) if(!visited[u]) dfs(u, g, out);
8a8     out.push_back(v);
65c }

c44 vector<v64> scc(vector<v64>& g) {
92e     int n = sz(g);
10e     v64 order, roots(n, 0);

ad4     vector<v64> adj_rev(n);
0d6     forn(u, 0, n) for (ll v : g[u]) adj_rev[v].push_back(u);

e18     visited.assign(n, false);
7bf     forn(i, 0, n) if (!visited[i]) dfs(i, g, order);
962     reverse(order.begin(), order.end());

e18     visited.assign(n, false);
a13     ll curr_comp = 0;
1a4     for (auto v : order) {
936         if (!visited[v]) {
304             v64 component; dfs(v, adj_rev, component);
11f             for (auto u : component) roots[u] = curr_comp;
b12             curr_comp++;
b46         }
bed     }

697     set<p64> edges;
5e8     vector<v64> cond_g(curr_comp);
4c4     forn(u, 0, n) {
45c         for (auto v : g[u]) {
a88             if (roots[u] != roots[v] && !edges.count({roots[u],
roots[v]})) {
bbd                 cond_g[roots[u]].push_back(roots[v]);
cf1                 edges.emplace(roots[u], roots[v]);
ee1             }
e33         }
9b3     }
325     return cond_g;
893 }
```

2.6 2-SAT 943

solve() retorna um par, o first fala se eh possivel atribuir, o second fala se cada variavel eh verdadeira O(|V|+|E|) = O(#variaveis + #restricoes)

```
e39 #include<bits/stdc++.h>
23e using namespace std;
```

```
ab0 typedef long long ll;
609 typedef vector<ll> v64;

ecf struct sat {
7fa     ll n, tot;
cf5     vector<v64> g;
83e     v64 vis, comp, id, ans;
9c5     stack<ll> s;

6d2     sat() {}
3c5     sat(ll n_) : n(n_), tot(n), g(2*n) {}

b9d     ll dfs(ll i, ll& t) {
ad4         ll lo = id[i] = t++;
381         s.push(i), vis[i] = 2;
2b0         for (ll j : g[i]) {
4e4             if (!vis[j]) lo = min(lo, dfs(j, t));
840             else if (vis[j] == 2) lo = min(lo, id[j]);
228         }
b84         if (lo == id[i]) while (1) {
e4a             ll u = s.top(); s.pop();
7ff             vis[u] = 1, comp[u] = i;
506             if ((u>>1) < n && ans[u>>1] == -1) ans[u>>1] = ~u&1;
56b             if (u == i) break;
20e         }
ad1         return lo;
3a7     }

5d5     void add_impl(ll x, ll y) { // x -> y = !x ou y
84f         x = x >= 0 ? 2*x : -2*x-1;
dc6         y = y >= 0 ? 2*y : -2*y-1;
ef5         g[x].push_back(y);
add         g[y^1].push_back(x^1);
b3c     }
e62     void add_cl(ll x, ll y) { // x ou y
935         add_impl(~x, y);
64b     }
00b     void add_xor(ll x, ll y) { // x xor y
8ab         add_cl(x, y), add_cl(~x, ~y);
ace     }
3f7     void add_eq(ll x, ll y) { // x = y
a3b         add_xor(~x, y);
68c     }
dbb     void add_true(ll x) { // x = T
956         add_impl(~x, x);
564     }
dbf     void at_most_one(v64 v) { // no max um verdadeiro
909         g.resize(2*(tot+sz(v)));
9fc         for (ll i = 0; i < sz(v); i++) {
33c             add_impl(tot+i, ~v[i]);
c82             if (i) {
a6f                 add_impl(tot+i, tot+i-1);
d24                 add_impl(v[i], tot+i-1);
586             }
7dc         }
136         tot += sz(v);
1a0     }

d26     pair<bool, v64> solve() {
f59         ans = v64(n, -1);
b4d         ll t = 0;
d68         vis = comp = id = v64(2*tot, 0);
96e         for (ll i = 0; i < 2*tot; i++) if (!vis[i]) dfs(i, t);
289         for (ll i = 0; i < tot; i++)
105             if (comp[2*i] == comp[2*i+1]) return {false, {}};
d78         return {true, ans};
8d8     }
1f2 };
```

2.7 Topological Sort (Kahn's Algorithm) 9b8

Produces a topological ordering of a DAG using indegree counting and a queue-like frontier.

Complexity: O(N + M), O(N)

```
a08 v64 topo_sort(const vector<v64>& g) {
d0d     v64 indeg(sz(g)), q;
635     for (auto& li : g) for (int x : li) indeg[x]++;
8da     forn(i, 0, sz(g)) if (indeg[i] == 0) q.push_back(i);
d61     forn(j, 0, sz(q)) for(int x : g[q[j]]) if(--indeg[x] == 0) q.
push_back(x);
2e8     return q;
9b8 }
```

2.8 LCA com binary lifting 7de

Assume que um vertice eh ancestral dele mesmo, ou seja, se a eh ancestral de b, lca(a, b) = a MAXLOG = ceil(log(MAX)) Complexidades: build - O(n log(n)) lca - O(log(n))

```
202 vector<v64> g(MAX);
7e0 ll n, p;
9e8 ll parent[MAXLOG][MAX];
c5e ll in[MAX], out[MAX];

fd3 void dfs(ll k) {
18f     in[k] = p++;
654     for(ll v: g[k]){
050         if(in[v] == -1){
e10             parent[0][v] = k;
a7e             dfs(v);
a67         }
3a5     }
c34     out[k] = p++;
ab4 }

bf8 void build(ll raiz) {
46c     forn(i,0,n) parent[0][i] = i;
014     p = 0, memset(in, -1, sizeof in);
6b5     dfs(raiz);
104     forn(k,1,MAXLOG) forn(i,0,n)
423         parent[k][i] = parent[k - 1][parent[k - 1][i]];
896 }

fb3 bool anc(ll a, ll b) { // se a eh ancestral de b
b01     return in[a] <= in[b] && out[a] >= out[b];
306 }

002 ll lca(ll a, ll b) {
8ff     if (anc(a, b)) return a;
828     if (anc(b, a)) return b;
084     for (ll k = MAXLOG - 1; k >= 0; k--)
5f1         if (!anc(parent[k][a], b)) a = parent[k][a];
6a3     return parent[0][a];
b4f }
```

2.9 Heavy Light Decomposition (Path query + Path update) 260

Decomposes a tree into vertex disjoint heavy paths and light edges such that the path from any leaf to the root contains at most log(n) light edges. All values initialized to the segtree default. Root must be 0. Uses a Lazy Segment Tree

Complexity: O((log N)$\hat{}$}2) per op, O(N) build

```
273 template <bool VALS_EDGES> struct HLD {
657     ll N, tim = 0;
02a     vector<v64> adj;
f53     v64 parent, siz, head, pos;
b32     vector<Node> vseg;
e73     std::unique_ptr<segtree<Node, Update>> seg;
ae4     HLD(vector<v64> adj_, v64 vals)
7c1         : N(sz(adj_)), adj(adj_), parent(N, -1), siz(N, 1),
1dd           head(N),pos(N),vseg(N, {0}){ dfsSz(0); dfsHld(0);
aec             seg = make_unique<segtree<Node, Update>>(N);
d8f             seg->set_leaves(vseg);
0b3         }
d5e     void dfsSz(ll v) { // get heavy son
d17         for (ll& u : adj[v]) {
ac6             adj[u].erase(find(adj[u].begin(), adj[u].end(), v));
762             parent[u] = v;
336             dfsSz(u);
ec7             siz[v] += siz[u];
1e2             if (siz[u] > siz[adj[v][0]]) swap(u, adj[v][0]);
561         }
bc8     }
016     void dfsHld(ll v) { // "linearizes" the tree
9e4         pos[v] = tim++;
ca5         for (ll u : adj[v]) {
63f             head[u] = (u == adj[v][0] ? head[v] : u);
8b3             dfsHld(u);
8ff         }
c03     }
ed1     template <class B> void process(ll u, ll v, B op) {
aea         for (;; v = parent[head[v]]) {
c30             if (pos[head[u]] > pos[head[v]]) swap(u, v);
152             if (head[u] == head[v]) break;
983             op(pos[head[v]], pos[v]);
82b         }
770         if (pos[u] > pos[v]) swap(u, v);
fd8         op(pos[u] + VALS_EDGES, pos[v]);
00e     }
546     void modifyPath(ll u, ll v, ll val) {
491         process(u, v, [&](ll l, ll r) {
65b
            seg->update(l, r, {val}); // Modify depending on problem
516         });
8b0     }
770     ll queryPath(ll u, ll v) { // Modify depending on problem
b3a         ll res = -INF;
491         process(u, v, [&](ll l, ll r) {
878                 res = max(res, seg->query(l, r).val);
5bc         });
8df         return res;
16d     }
20d     ll querySubtree(ll v) { // modifySubtree is similar
e8c         return seg->query(pos[v] + VALS_EDGES, pos[v] + siz[v] -
 1).val;
130     }
260 };
```

2.10 Centroid Decomposition - pares a distancia k 67a

Objetivo: contar #pares (u,v) com dist(u,v)=k na arvore. Ideia: em cada centroide c, para cada subarvore j: - path = profundidades d partindo de j (sem nos removidos) - somar cnt[k-d-1] ANTES de atualizar (evita dupla contagem entre subarvores) - depois, cnt[d+1]++ (a aresta (c,j) soma 1) Funcoes: dfs_sz (tamanhos), centroid (acha centroide), dfs (coleta depths), decomp (divide e combina). Complexidade: O(n log n) tempo; O(n) memoria. Cuidados: limites de cnt, k==0 trata a parte se quiser contar (u,u); rem[c] permanece marcado. escrito pelo gpt, cuidado

```
978 vector<ll> g[MAX];
a00 ll sz[MAX], rem[MAX];

32f void dfs(v64& path, ll i, ll l=-1, ll d=0) {
bc8     path.push_back(d);
ca6
    for (ll j : g[i]) if (j != l && !rem[j]) dfs(path, j, i, d+1);
510 }

651 ll dfs_sz(ll i, ll l=-1) {
0a8     sz[i] = 1;
1c7
    for (ll j : g[i]) if (j != l && !rem[j]) sz[i] += dfs_sz(j, i);
4a0     return sz[i];
a79 }

054 ll centroid(ll i, ll l, ll size) {
894     for (ll j : g[i]) if (j != l && !rem[j] && sz[j] > size / 2)
9a5         return centroid(j, i, size);
b36     return i;
2a5 }

23e ll decomp(ll i, ll k) {
87f     ll c = centroid(i, i, dfs_sz(i));
437     rem[c] = 1;

     // gasta O(n) aqui - dfs sem ir pros caras removidos
0a3     ll ans = 0;
419     vector<ll> cnt(sz[i]);
5fb     cnt[0] = 1;
e27     for (ll j : g[c]) if (!rem[j]) {
b53         vector<ll> path;
70a         dfs(path, j);
116         for (ll d : path) if (0 <= k-d-1 && k-d-1 < sz[i])
1fc             ans += cnt[k-d-1];
bc0         for (ll d : path) cnt[d+1]++;
5bb     }

244     for (ll j : g[c]) if (!rem[j]) ans += decomp(j, k);
694     rem[c] = 0;
ada     return ans;
840 }
```

3 Math

3.1 Modular Integer 123

*Fixed-modulus integer type with +, -, *, /, and exponentiation; modulo should be prime for division via Fermat.*

Complexity: O(1) per arithmetic op (O(log E) for exponentiation), O(1)

```
335 const ll MOD = 998244353;

7a5 template<ll p> struct mod_int {
d27     ll expo(ll b, ll e) {
2b3         ll ret = 1;
382         while (e) {
9dc             if (e % 2) ret = ret * b % p;
f84             e /= 2, b = b * b % p;
30f         }
3cb         return ret;
931     }
7c0     ll inv(ll b) { return expo(b, p-2); }

591     using m = mod_int;
da9     ll v;
68b     mod_int() : v(0) {}
89c     mod_int(ll v_) {
323         if (v_ >= p || v_ <= -p) v_ %= p;
95b         if (v_ < 0) v_ += p;
6b4         v = v_;
3ac     }
e8a     m& operator +=(const m& a) {
385         v += a.v;
2f4         if (v >= p) v -= p;
ace         return *this;
2e9     }
681     m& operator -=(const m& a) {
cc9         v -= a.v;
1db         if (v < 0) v += p;
ace         return *this;
b87     }
f5d     m& operator *=(const m& a) {
717         v = v * a.v % p;
ace         return *this;
5d9     }
5ae     m& operator /=(const m& a) {
5b9         v = v * inv(a.v) % p;
ace         return *this;
b8b     }
1fd     m operator -() const { return m(-v); }
188     m& operator ^=(ll e) {
a64         if (e < 0) {
639             v = inv(v);
1e0             e = -e;
056         }
3b5         v = expo(v, e);
         // possible optimization:
         // careful with 0^0
         // v = expo(v, e%(p-1));
ace         return *this;
17e     }
c01     bool operator ==(const m& a) { return v == a.v; }
9b6     bool operator !=(const m& a) { return v != a.v; }

876     friend istream& operator >>(istream& in, m& a) {
13b         ll val; in >> val;
9e8         a = m(val);
701         return in;
6c3     }
9fe     friend ostream& operator <<(ostream& out, m a) {
8a9         return out << a.v;
55b     }
bc1     friend m operator +(m a, m b) { return a += b; }
656     friend m operator -(m a, m b) { return a -= b; }
fd6     friend m operator *(m a, m b) { return a *= b; }
d6f     friend m operator /(m a, m b) { return a /= b; }
c67     friend m operator ^(m a, ll e) { return a ^= e; }
c07 };
47b typedef mod_int<MOD> mint;
```

3.2 Modular Arithmetic Helpers 042

Provides modular add/sub/mul, fast exponentiation, and modular inverse under fixed MOD.

Complexity: O(log E) for power/inverse

```
112 const ll MOD = 1_000_000_007;

490 inline ll sum(ll a, ll b) { a += b; if (a >= MOD) a -= MOD;
return a; }
fd6 inline ll sub(ll a, ll b) { a -= b; if (a < 0)  a += MOD; return
 a; }
bc6 inline ll mult(ll a, ll b) { return (a * b) % MOD; }

eaa inline ll pot(ll base, ll exp) {
0c6     ll res = 1;
89a     while (exp) {
da8         if (exp & 1) res = mult(res, base);
066         base = mult(base, base);
cef         exp >>= 1;
613     }
8df     return res;
594 }

bb5 inline ll inv_mod(ll a) {return pot(a, MOD-2);}
```

3.3 Linear Sieve (prime precomputation) ef5

Computes primes up to N in O(N) using the smallest prime factor (lp). pr will contain all primes <= N. lp[x] stores the smallest prime dividing x. escrito pelo gpt, cuidado

Complexity: O(N)

```
70d const ll N = 10000000;
4ee v64 lp(N+1), pr;

b60 void sieve(){
b67     forn(i, 2, N+1) {
d68         if (lp[i] == 0) {
98a             lp[i] = i;
e0d             pr.push_back(i);
b04         }
4bc         for (ll j = 0; i * pr[j] <= N; ++j) {
97c             lp[i * pr[j]] = pr[j];
998             if (pr[j] == lp[i]) {
c9a                 break;
963             }
c74         }
d9d     }
2ef }
```

3.4 FFT/NTT Convolution d95

Implements iterative FFT over complex numbers and NTT over supported primes; provides convolution utility.

Complexity: O(N log N), O(N)

```
8ac void get_roots(bool f, int n, vector<complex<double>>& roots) {
679     const static double PI = acosl(-1);
96d     for (int i = 0; i < n/2; i++) {
8e4         double alpha = i*((2*PI)/n);
233         if (f) alpha = -alpha;
4b4         roots[i] = {cos(alpha), sin(alpha)};
```

```
1e6     }
e85 }

// Para NTT
65a template<int p>
8ab void get_roots(bool f, int n, vector<mod_int<p>>& roots) {
47f     mod_int<p> r;
df5     int ord;
2c5     if (p == 998244353) {
620         r = 102292;
350         ord = (1 << 23);
a9f     } else if (p == 754974721) {
530         r = 739831874;
00d         ord = (1 << 24);
f8d     } else if (p == 167772161) {
13a         r = 243;
84e         ord = (1 << 25);
3c2     } else assert(false);

e6b     if (f) r = r^(p - 1 -ord/n);
1a7     else r = r^(ord/n);
dcd     roots[0] = 1;
5ea     for (int i = 1; i < n/2; i++) roots[i] = roots[i-1]*r;
892 }

27e template<typename T> void fft(vector<T>& a, bool f, int N,
vector<int>& rev) {
179     for (int i = 0; i < N; i++) if (i < rev[i]) swap(a[i], a[rev
[i]]);
4a1     int l, r, m;
a38     vector<T> roots(N);
66d     for (int n = 2; n <= N; n *= 2) {
13c         get_roots(f, n, roots);
017         for (int pos = 0; pos < N; pos += n) {
235             l = pos + 0, r = pos + n/2, m = 0;
0a7             while (m < n/2) {
451                 auto t = roots[m] * a[r];
54c                 a[r] = a[l] - t;
db7                 a[l] = a[l] + t;
c26                 l++, r++, m++;
a3a             }
db8         }
925     }
80d     if (f) {
dd9         auto invN = T(1) / T(N);
667         for (int i = 0; i < N; i++) a[i] = a[i] * invN;
d42     }
a5b }

9f6 template<typename T> vector<T> convolution(vector<T>& a, vector<
T>& b) {
902     vector<T> l(a.begin(), a.end()), r(b.begin(), b.end());
927     int N = sz(l)+sz(r)-1;
aea     int n = 1, log_n = 0;
884     while (n <= N) n *= 2, log_n++;
dea     vector<int> rev(n);
8ee     for (int i = 0; i < n; i++) {
23e         rev[i] = 0;
f95         for (int j = 0; j < log_n; j++) if (i>>j&1)
2be             rev[i] |= 1 << (log_n-1-j);
046     }

2f0     assert(N <= n);
5c1     l.resize(n);
ca3     r.resize(n);
769     fft(l, false, n, rev);
4eb     fft(r, false, n, rev);
186     for (int i = 0; i < n; i++) l[i] *= r[i];
99d     fft(l, true, n, rev);
758     l.resize(N);
273     return l;
55a }

// NTT
b09 template<int p, typename T>
ebc vector<mod_int<p>> ntt(vector<T>& a, vector<T>& b) {
3b0
    vector<mod_int<p>> A(a.begin(), a.end()), B(b.begin(), b.end());
697     return convolution(A, B);
32b }
```

3.5 Matrix template 720

- operator: multiplicacao (O(n*m*r)) opcional modo modular (#define MODULAR true) - operator{}: exponenciacao rapida (O(log e)) - apply_transform(): aplica transformacao M{}e * v*

```
205 #define MODULAR false
692 template<typename T> struct matrix : vector<vector<T>> {
908     ll n, m;
f23     void print() {
bda         forn(i,0,n) {
80b             forn(j,0,m) cout << (*this)[i][j] << " ";
8a6             cout << endl;
aeb         }
40e     }

e9c     matrix(ll n_, ll m_, bool ident = false) :
b03
            vector<vector<T>>(n_, vector<T>(m_, 0)), n(n_), m(m_) {
1ed         if (ident) {
e7b             assert(n == m);
e7f             forn(i,0,n) (*this)[i][i] = 1;
122         }
a86     }
29c     matrix(const vector<vector<T>>& c) : vector<vector<T>>(c),
133         n(sz(c)), m(sz(c[0])) {}
1b5     matrix(const initializer_list<initializer_list<T>>& c) {
7dc         vector<vector<T>> val;
de7         for (auto& i : c) val.push_back(i);
cec         *this = matrix(val);
f06     }

f86     matrix<T> operator*(matrix<T>& r) {
43c         assert(m == r.n);
c2e         matrix<T> M(n, r.m);
966         forn(i,0,n) forn(k,0,m) forn(j, 0, r.m){
bc4             T add = (*this)[i][k] * r[k][j];
f98 #if MODULAR
8d4 #warning Usar matrix<ll> e soh colocar valores em [0, MOD) na
matriz!
ba1             M[i][j] += add%MOD;
795             if (M[i][j] >= MOD) M[i][j] -= MOD;
8c1 #else
3f3             M[i][j] += add;
f2e #endif
5bb         }
3f1         return M;
2de     }
2e3     matrix<T> operator^(ll e){
6ff         matrix<T> M(n, n, true), at = *this;
382         while (e) {
92d             if (e&1) M = M*at;
133             e >>= 1;
5ef             at = at*at;
b1a         }
3f1         return M;
84d     }
f49     void apply_transform(matrix M, ll e){
dd9         auto& v = *this;
382         while (e) {
394             if (e&1) v = M*v;
133             e >>= 1;
cd1             M = M*M;
01f         }
660     }
955 };
```

3.6 Gauss 7a5

*Resolve sistema linear Retornar um par com o numero de solucoes e alguma solucao, caso exista O(n{}2 * m) escrito pelo gpt, cuidado*

```
442 template<typename T>
0b9 pair<ll, vector<T>> gauss(vector<vector<T>> a, vector<T> b) {
ef1     const double eps = 1e-6;
337     ll n = sz(a), m = sz(a[0]);
fcb     forn(i, 0, n) a[i].push_back(b[i]);

af0     v64 where(m, -1);
afc     for (ll col = 0, row = 0; col < m && row < n; col++) {
caa         ll sel = row;
ed1         forn(i, row, n)
4c9             if(abs(a[i][col]) > abs(a[sel][col])) sel = i;
912         if(abs(a[sel][col]) < eps) continue;
63f         forn(i, col, m+1)
554             swap(a[sel][i], a[row][i]);
a32         where[col] = row;

c62         forn(i, 0, n) if (i != row) {
5f0             T c = a[i][col] / a[row][col];
36d             forn(j, col, m+1)
c18                 a[i][j] -= a[row][j] * c;
e08         }
42f         row++;
2fd     }

88e     vector<T> ans(m, 0);
b49     forn(i, 0, m) if (where[i] != -1)
825         ans[i] = a[where[i]][m]/a[where[i]][i];
bda     forn(i, 0, n) {
655         T sum = 0;
96b         forn(j, 0, m)
345             sum += ans[j]*a[i][j];
d72         if(abs(sum - a[i][m]) > eps)
63a             return pair(0, vector<T>());
db1     }

445     forn(i, 0, m) if (where[i] == -1)
335         return pair(INF, ans);
abd     return pair(1, ans);
```

```
15a }
```

4 Strings

4.1 Knuth-Morris-Pratt (KMP) 514

Computes prefix function and performs linear-time substring search with optional automaton construction.

Complexity: O(n + m), O(n)

```
7f8 v64 pi(string& s) {
761     v64 p(sz(s));
655     for (ll i = 1, j = 0; i < sz(s); i++) {
407         while (j && s[j] != s[i]) j = p[j-1];
1c1         if (s[j] == s[i]) j++;
6fb         p[i] = j;
314     }
4e5     return p;
3fb }

158 v64 match(string& pat, string& s) {
086     v64 p = pi(pat), match;
4e5     for (ll i = 0, j = 0; i < sz(s); i++) {
d77         while (j && pat[j] != s[i]) j = p[j-1];
c8b         if (pat[j] == s[i]) j++;
1b1         if (j == sz(pat)) match.push_back(i-j+1), j = p[j-1];
f18     }
de3     return match;
982 }

1c8 struct KMPaut : vector<v64> {
ff6     KMPaut(){}
29d     KMPaut (string& s) : vector<v64>(26, v64(sz(s)+1)) {
3f3         v64 p = pi(s);
b29         auto& aut = *this;
c31         aut[s[0]-'a'][0] = 1;
50a         for (char c = 0; c < 26; c++)
2bf             for (int i = 1; i <= sz(s); i++)
1a8
                aut[c][i] = s[i]-'a' == c ? i+1 : aut[c][p[i-1]];
4a6     }
3b4 };
```

4.2 Aho-Corasick Automaton 01f

Builds a trie with failure links for multi-pattern matching; insert is O(|s|), build is linear in total length, and queries run in linear time in the text.

Complexity: varies, O(total patterns length)

```
427 namespace aho {
7bc     map<char, ll> to[MAX];
452     ll link[MAX], idx, term[MAX], exit[MAX], sobe[MAX];
db4     vector<ll> max_match(MAX, 0);

c0c     void insert(string& s) {
9a3         ll at = 0;
419         for (char c : s) {
5b1             auto it = to[at].find(c);
b64             if (it == to[at].end()) at = to[at][c] = ++idx;
b2d             else at = it->second;
44e         }
2ba         term[at]++, sobe[at]++;
5f8         max_match[at] = sz(s);
b7a     }

91f     void build() {
567         queue<ll> q;
b47         q.push(0);
478         link[0] = exit[0] = -1;
dad         while (sz(q)) {
950             ll i = q.front(); q.pop();
df1             for (auto [c, j] : to[i]) {
1bd                 ll l = link[i];
657                 while (l != -1 && !to[l].count(c)) l = link[l];
a68                 link[j] = l == -1 ? 0 : to[l][c];
546
                exit[j] = term[link[j]] ? link[j] : exit[link[j]];

02b                 max_match[j] = max(max_match[link[j]], max_match
[j]);
fb8                 if (exit[j]+1) sobe[j] += sobe[exit[j]];
df8                 q.push(j);
a97             }
e9c         }
555     }
3b5     ll query(string& s) {
a33         ll at = 0, ans = 0;
419         for (char c : s){
bae             while (at != -1 && !to[at].count(c)) at = link[at];
2ab             at = at == -1 ? 0 : to[at][c];
56a             ans += sobe[at];
deb         }
ada         return ans;
c01     }
078     vector<ll> match_vec(string& s) {
fec         ll at = 0, n = sz(s);
ccc         vector<ll> v(n, 0);
bda         forn(i, 0, n){
aa0             char c = s[i];
bae             while (at != -1 && !to[at].count(c)) at = link[at];
2ab             at = at == -1 ? 0 : to[at][c];

ab5             v[i] = max_match[at]; // quero isso
63e         }
f1e         return v;
7a7     }
01f }
```

4.3 Suffix Array - O(n log n) bfc

kasai recebe o suffix array e calcula lcp[i], o lcp entre s[sa[i],...,n-1] e s[sa[i+1],..,n-1] Complexidades: suffix_array - O(n log(n)) kasai - O(n)

```
c73 v64 suffix_array(string s) {
3c2     s.push_back('$'); // 0 caso v64 (CHECAR SE PODE)
0ce     ll n = sz(s), N = max(n, 260ll);
bb4     v64 sa(n), ra(n);
c15     forn(i, 0, n) sa[i] = i, ra[i] = s[i];

d0d     for(ll k = 0; k < n; k ? k *= 2 : k++) {
b35         v64 nsa(sa), nra(n), cnt(N);

dff         for(int i = 0; i < n; i++) nsa[i] = (nsa[i]-k+n)%n, cnt[
ra[i]]++;

20e         forn(i, 1, N) cnt[i] += cnt[i-1];
ece
        for(ll i = n-1; i+1; i--) sa[--cnt[ra[nsa[i]]]] = nsa[i];

034         for(ll i = 1, r = 0; i < n; i++) nra[sa[i]] = r += ra[sa
[i]] !=
ce2             ra[sa[i-1]] || ra[(sa[i]+k)%n] != ra[(sa[i-1]+k)%n];
b03         ra = nra;
666         if (ra[sa[n-1]] == n-1) break;
969     }
054     return v64(sa.begin()+1, sa.end());
69d }

7a6 v64 kasai(string s, v64 sa) {
645     ll n = sz(s), k = 0;
ce4     v64 ra(n), lcp(n);
9c3     forn(i, 0, n) ra[sa[i]] = i;

b9d     for (ll i = 0; i < n; i++, k -= !!k) {
c37         if (ra[i] == n-1) { k = 0; continue; }
8ba         ll j = sa[ra[i]+1];
443         while (i+k < n && j+k < n && s[i+k] == s[j+k]) k++;
840         lcp[ra[i]] = k;
bd0     }
8ad     return lcp;
60f }
```

5 Dynamic Programming

5.1 Divide and Conquer DP Optimization 1b1

Optimizes DP transitions with quadrangle inequality/monge-like structure using divide-and-conquer over optimal decision points.

Complexity: O(K N log N) with O(1) cost, O(N)

```
4d6 vector<v64> dp; // dp[n+1][2]

afe void solve(ll k, ll l, ll r, ll lk, ll rk) {
bd4     if (l > r) return;
775     ll m = (l+r)/2, p = -1;
66f     auto& ans = dp[m][k&1] = INF;
c8a     for (ll i = max(m, lk); i <= rk; i++) {
389         ll at = dp[i+1][~k&1] + cost(m, i);
19b         if (at < ans) ans = at, p = i;
e3c     }
129     solve(k, l, m-1, lk, p), solve(k, m+1, r, p, rk);
c8f }

8c3 ll dnc(ll n, ll k) {
b45     dp[n][0] = dp[n][1] = 0;
925     forn(i,0,n) dp[i][0] = INF;
157     forn(i,1,k+1) solve(i, 0, n-i, 0, n-i);
b9c     return dp[0][k&1];
aa3 }
```

5.2 Sum over Subsets DP (SOS DP) 773

sos_dp_sub: for each mask, computes the sum of f[s] over all subsets s of mask (i.e., f'[mask] = sum of f[s] for s in mask). sos_dp_super: for each mask, computes the sum of f[s] over all supersets s of mask (i.e., f'[mask] = sum of f[s] for mask in s). input size must be a power of two (2^n).

Complexity: O(n * 2^n), O(2^n) mem

```
66a v64 sos_dp_sub(v64 f) {
848     ll n = __builtin_ctzll(sz(f));
77f     assert((1ll<<n) == sz(f));
```

```
00d     forn(i,0,n) forn(mask, 0, (1<<n))
ec7         if (mask>>i&1) f[mask] += f[mask^(1<<i)];
         // -= for inverse
f41     return f;
e2d }

69d v64 sos_dp_super(v64 f) {
848     ll n = __builtin_ctzll(sz(f));
77f     assert((1ll<<n) == sz(f));

00d     forn(i,0,n) forn(mask, 0, (1<<n))
22b         if (~mask>>i&1) f[mask] += f[mask^(1<<i)];
         // -= for inverse
f41     return f;
be7 }
```

6 Geometry

6.1 Integer Geometry Primitives 001

Defines 2D point and line structures with orientation, area, and angle comparisons plus a sweep-line comparator.

```
c01 struct pt { // point
57c     ll x, y;
8cc     pt(ll x_ = 0, ll y_ = 0) : x(x_), y(y_) {}
537     bool operator < (const pt p) const {
dac         if (x != p.x) return x < p.x;
2d1         return y < p.y;
f98     }
652     bool operator == (const pt p) const {
4f6         return x == p.x && y == p.y;
c9e     }
377
    pt operator + (const pt p) const { return pt(x+p.x, y+p.y); }
e45
    pt operator - (const pt p) const { return pt(x-p.x, y-p.y); }
953     pt operator * (const ll c) const { return pt(x*c, y*c); }
881     ll operator * (const pt p) const { return x*(ll)p.x + y*(ll)
p.y; } // dot product
f4f     ll operator ^ (const pt p) const { return x*(ll)p.y - y*(ll)
p.x; } // cross product
97f     friend istream& operator >> (istream& in, pt& p) {
7cf         return in >> p.x >> p.y;
3a6     }
d91 };

e42 struct line { // line segment
ca5     pt p, q;

9d3     line() {}
9b5     line(pt p_, pt q_) : p(p_), q(q_) {}
fe1     bool operator < (const line l) const {
d8d         if (!(p == l.p)) return p < l.p;
46b         return q < l.q;
887     }
935     bool operator == (const line l) const {
c63         return p == l.p && q == l.q;
485     }
410     friend istream& operator >> (istream& in, line& r) {
def         return in >> r.p >> r.q;
2c5     }
ead };
```

```
c60 ll sarea2(pt p, pt q, pt r) { // 2 * signed area
411     return (q-p)^(r-q);
74b }

9ad ll norm_sq(pt p){ return p*p;}

5b9 ll dist_sq(pt p, pt q){ return norm_sq(p - q);}

d15 bool ccw(pt p, pt q, pt r) { // true if p, q, r are
236     return sarea2(p, q, r) > 0;
85f }

b9f int quad(pt p) { // quadrant of a point
472     return (p.x<0)^3*(p.y<0);
09a }

6f3 bool compare_angle(pt p, pt q) { // returns true if angle(p) <
angle(q)
591     if (quad(p) != quad(q)) return quad(p) < quad(q);
74f     return ccw(q, pt(0, 0), p);
b1c }

ae5 bool isinseg(pt p, line r) { // se p pertence ao seg de r
ee2     pt a = r.p - p, b = r.q - p;
c3c     return (a ^ b) == 0 && (a * b) <= 0;
7c8 }

a74 bool interseg(line r, line s) { // se o seg de r intersecta o
seg de s
8dd     if (isinseg(r.p, s) || isinseg(r.q, s)
d30         || isinseg(s.p, r) || isinseg(s.q, r)) return 1;

4b8     return ccw(r.p, r.q, s.p) != ccw(r.p, r.q, s.q) &&
7a3             ccw(s.p, s.q, r.p) != ccw(s.p, s.q, r.q);
1be }

806 ll polarea(vector<pt> v) { // area do poligono
125     ll ret = 0;
2d0     forn(i, 0, sz(v))
49f         ret += sarea2(pt(0, 0), v[i], v[(i + 1) % sz(v)]);
ba7     return abs(ret);
80a }

// comparator for set to perform sweep line with segments
ca2 struct cmp_sweepline {
553     bool operator () (const line& a, const line& b) const {
         // assumes segments have p < q
f8c         if (a.p == b.p) return ccw(a.p, a.q, b.q);
72e         if (a.p.x != a.q.x && (b.p.x == b.q.x || a.p.x < b.p.x))
b87             return ccw(a.p, a.q, b.p);
c35         return ccw(a.p, b.q, b.p);
cae     }
fc1 };
```

6.2 Float Geometry Primitives 507

Defines 2D point and line structures with orientation, area, and angle comparisons plus a sweep-line comparator.

```
17e typedef long double ld;

b47 const ld DINF = 1e18;
3b5 const ld eps = 1e-9;
45e const ld pi = 2*acos(0);
```

```
c6e #define sq(x) ((x)*(x))

1fc ll cmp(ld x, ld y = 0, ld tol = eps) {
534     return (x <= y + tol) ? (x + tol < y) ? -1 : 0 : 1;
49f }

c01 struct pt { // ponto
2d5     ld x, y;
292     pt(ld x_ = 0, ld y_ = 0) : x(x_), y(y_) {}
537     bool operator < (const pt p) const {
ccf         if (cmp(x, p.x) != 0) return cmp(x, p.x) < 0;
6c8         if (cmp(y, p.y) != 0) return cmp(y, p.y) < 0;
ca4         return 0;
554     }
652     bool operator == (const pt p) const {
580         return cmp(x, p.x) == 0 && cmp(y, p.y) == 0;
6df     }
377
    pt operator + (const pt p) const { return pt(x+p.x, y+p.y); }
e45
    pt operator - (const pt p) const { return pt(x-p.x, y-p.y); }
4e0
    pt operator * (const ld c) const { return pt(x*c  , y*c  ); }
3aa
    pt operator / (const ld c) const { return pt(x/c  , y/c  ); }
1c3     ld operator * (const pt p) const { return x*p.x + y*p.y; }
59b     ld operator ^ (const pt p) const { return x*p.y - y*p.x; }
97f     friend istream& operator >> (istream& in, pt& p) {
7cf         return in >> p.x >> p.y;
3a6     }
759 };

e42 struct line { // reta
ca5     pt p, q;
9d3     line() {}
9b5     line(pt p_, pt q_) : p(p_), q(q_) {}
410     friend istream& operator >> (istream& in, line& r) {
def         return in >> r.p >> r.q;
2c5     }
4f9 };

4a5 ld norm_sq(pt p){ return p*p;}

8cb ld dist_sq(pt p, pt q){ return norm_sq(p - q);}

3ea ld dist(pt p,pt q){ return sqrt(dist_sq(p, q));}

3c7 ld angle(pt v) { // angulo do vetor com o eixo x
d57     ld ang = atan2(v.y, v.x);
c65     if (cmp(ang, 0) < 0) ang += 2*pi;
dbd     return ang;
b17 }

beb ld sarea(pt p, pt q, pt r) { // area com sinal
f6e     return ((q-p)^(r-q))/2;
9cd }

093 bool col(pt p, pt q, pt r) { // se p, q e r sao colin.
f02     return cmp(sarea(p, q, r), 0) == 0;
495 }

d15 bool ccw(pt p, pt q, pt r) { // se p, q, r sao ccw
```

```
69d
    return cmp(sarea(p, q, r),0) > 0; // talvez vc queira = !!!!!!!
d05 }

a4d ld polarea(vector<pt> v) { // area do poligono
5d8     ld ret = 0;
2d0     forn(i, 0, sz(v))
65d         ret += sarea(pt(0, 0), v[i], v[(i + 1) % sz(v)]);
ba7     return abs(ret);
d71 }

4f8
ld get_t(pt v, line r) { // retorna t tal que t*v pertence a reta r
22e     return (r.p^r.q) / ((r.p-r.q)^v);
114 }

9d5 pt inter(line r, line s) { // r inter s
a3d     if (cmp((r.p - r.q) ^ (s.p - s.q), 0) == 0) return pt(DINF,
DINF);
7c8     r.q = r.q - r.p, s.p = s.p - r.p, s.q = s.q - r.p;
d1a     return r.q * get_t(r.q, s) + r.p;
50b }

// Intersecções de circulos

d3a pair<pt,pt> perp_pts(line l, ld d){ // retorna dois pontos r,s
tq rq e sq sao perp a pq e |rq| = |sq| = d;
22c     pt dir = l.p - l.q;
f3e     pt r = pt(dir.y, -dir.x);
501     pt k = r * (d/norm(r));
926     return {l.q + k, l.q - k};
63c }

827 pair<pt,pt> circint(pt o1, ld r1, pt o2, ld r2){
b73     pt dir = o2 - o1;
4e4     pt in(INFINITY, INFINITY);
a15     d = norm(dir);
    // tangent
58b     if(cmp(r1 + r2, d) == 0 || cmp(d + r2, r1) == 0 || cmp(d +
r1, r2) == 0)
a45         return {o1 + dir * r1/(r1 + r2), in};
    // no inter
58c     if(cmp(r1 + r2, d) == -1 || cmp(d + r2, r1) == -1 || cmp(d +
 r1, r2) == -1)
e03         return {in, in};
    // two inters
98f     ld x = (d*d - r2*r2 + r1*r1)/(2*d);
4c7     return disppt(o1, o1 + dir * x/d, sqrt(r1*r1 - x*x));
bfd }

f51 pair<pt,pt> circline(pt o, ld r, pt a, pt b){
2d2     pt h = foot(o, a, b);
ea2     pt in(DINF, DINF);
38c     ld d = norm(h - o);
3b9     if(cmp(d, r) == 0) return ppt{h, in};
c18     if(cmp(d, r) == 1) return ppt{in, in};
d97     return perp_pts(o, h, sqrt(r*r - d*d));
e5c }
```

6.3 Convex Hull (Monotone Chain) 23f

Computes the convex hull of a set of points using Andrew's monotone chain; handles collinear points based on ccw condition.

Complexity: O(N log N), O(N)

```
972 vector<pt> convex_hull(vector<pt> &v) {
cd9     v.erase(unique(v.begin(), v.end()), v.end());
6f2     if(sz(v) <=1) return v;
9aa     vector<pt> l, u;
e1f     forn(i, 0, sz(v)) {
986         while(sz(l) > 1 && !ccw(l.end()[-2], l.end()[-1], v[i]))
27f             l.pop_back();
482         l.push_back(v[i]);
a9a     }
e65     for(ll i = sz(v)-1; i>=0; i--) {
dfb         while(sz(u) > 1 && !ccw(u.end()[-2], u.end()[-1], v[i]))
74d             u.pop_back();
73b         u.push_back(v[i]);
d4e     }
27f     l.pop_back();
74d     u.pop_back();
106     for(pt i : u) l.push_back(i);
273     return l;
23f }
```

6.4 Halfplane Intersection 392

Acha o ponto mais extremo na direção dada só entra retas nao verticais, processar as retas verticais antes e encodar essa info no minx e maxx se quiser o ponto mais acima, fazer best_dir = (0,1) dependendo do best_dir, best = (+-INF, +-INF)

```
7b5 mt19937_64 rng((ll) chrono::steady_clock::now().time_since_epoch
().count());

fa6 pt hpi_1d(vector<line> v, pt best_dir, ll n, line suport, ll
minx, ll maxx){
020     pt low = inter(line(pt(minx, 0), pt(minx,1)), suport), upp =
 inter(line(pt(maxx, 0), pt(maxx,1)), suport);
bda     forn(i, 0, n){
331         pt intersection = inter(v[i], suport);
f09         debug(low);
3fc         debug(upp);
801         debug(intersection);
ebe         debug(v[i]);

bee         if(!ccw(v[i].p, v[i].q, low)) low = intersection;
d66         else if(!ccw(v[i].p, v[i].q, upp)) upp = intersection;
5bd     }

321     if(cmp(low.x, upp.x) > 0) return {DINF, DINF};

bcd     if(cmp(best_dir*low, best_dir*upp) <= 0) return upp;
c60     return low;
82a }

d04 pt hpi(vector<line> v, pt best_dir, ll minx, ll maxx){
efa     shuffle(v.begin(), v.end(), rng);

3d3     ll n = sz(v);
559     pt best(1, 1);

bda     forn(i, 0, n){
54d         debug(best);
ebe         debug(v[i]);
d42         if(ccw(v[i].p, v[i].q, best)) continue;
894         best = hpi_1d(v, best_dir, i, v[i], minx, maxx);
a9f         assert(!(best ==pt(DINF, DINF)));
269     }

5cb     return best;
993 }
```

7 Extra

7.1 template 3a8

```
e39 #include <bits/stdc++.h>
23e using namespace std;

ab0 typedef long long ll;
9a1 typedef pair<ll, ll> p64;
609 typedef vector<ll> v64;

3e7 #define forn(i, s, e) for (ll i = (s); i < (e); i++)
196 #define sz(u) ((ll) u.size())
e18 #define ln "\n"

ba0 #ifdef DEBUG
4f6 #define trace(u) u
5cb #define _
8c1 #else
073 #define trace(u)
358 #define _ ios::sync_with_stdio(0); cin.tie(0)
f2e #endif

bf4 #define debug(u) trace(cout << #u " = " << u << ln)
ca0 #define debugv(v) trace(cout << #v ": "; for (auto xx : v) cout
<< xx << " "; cout << ln)

4e7 const ll INF = 0x3f3f3f3f3f3f3f3fll;

8c2 int main() {
098     _;
ca4     return 0;
dce }
```

7.2 Pragmas acb

```
481 #pragma GCC optimize("O3,unroll-loops,fast-math")
941 #pragma GCC optimize("Ofast,fast-math,unroll-loops,inline")
c19 #pragma GCC optimize("Os")
```